SPRINT 01, FUNDAMENTOS DO PROCESSO

Ciclo de vida de software

SDLC, ALM, SCM e a malha CNCF: o processo que transforma código em produto que entrega valor.

Aula	01 de 14
Data	23 de abril de 2026, quarta-feira, 10h00 às 12h00
Professor	José Romualdo da Costa Filho
Módulo	13, Engenharia de Software, Turma T13

Agenda da aula

inteli.

Bloco	Atividade	Duração
10h00, Daily Time	Abertura com cronômetro de 15 min, status rápido por grupo	15 min
10h15, Ativação	Recuperação dos auto-estudos em quiz individual e plenária	25 min
10h40, Fundamentos	SDLC, metodologias, papéis, ALM, pipeline, ambientes, containers, Kubernetes, governança e OpenChoreo	35 min
11h15, Caso Jacto	Jacto Drones com três frentes sem processo unificado, prática em grupos	35 min
11h50, Síntese	Quiz de fechamento, homework e ponte para SCM	10 min

DAILY, 15 MIN

Daily Time

Alinhamento rápido de cada grupo: o que avançou desde a última aula, o que trava e qual é a próxima meta.

15:00

INICIAR PAUSAR ZERAR

Objetivos de aprendizagem

inteli.

Compreender

Reconhecer as fases do SDLC e a diferença entre SDLC, ALM e SCM.

Comparar

Distinguir cascata e ágil como duas formas de organizar o ciclo.

Situar

Posicionar o ecossistema CNCF e os cenários on-premises e cloud.

Aplicar

Mapear o ciclo das três frentes da Jacto Drones e propor um processo unificado.

BLOCO 1, 35 MIN

SDLC, metodologias, papéis e ecossistema

Software não começa no repositório e não termina no deploy. Vamos olhar o ciclo todo.

Software como processo

Ponto de partida

Antes do primeiro commit existe um problema de negócio, pessoas afetadas e uma hipótese. Depois do deploy existe operação, suporte, métrica, dívida técnica e novas versões. **Código é parte do ciclo, não o ciclo todo.**

Sem processo

- A decisão de hoje desfaz a de ontem. Retrabalho vira rotina.
- Ninguém sabe ao certo o que está em produção.
- Cada dev tem a sua convenção. O ritmo do time fica irregular.

Com processo

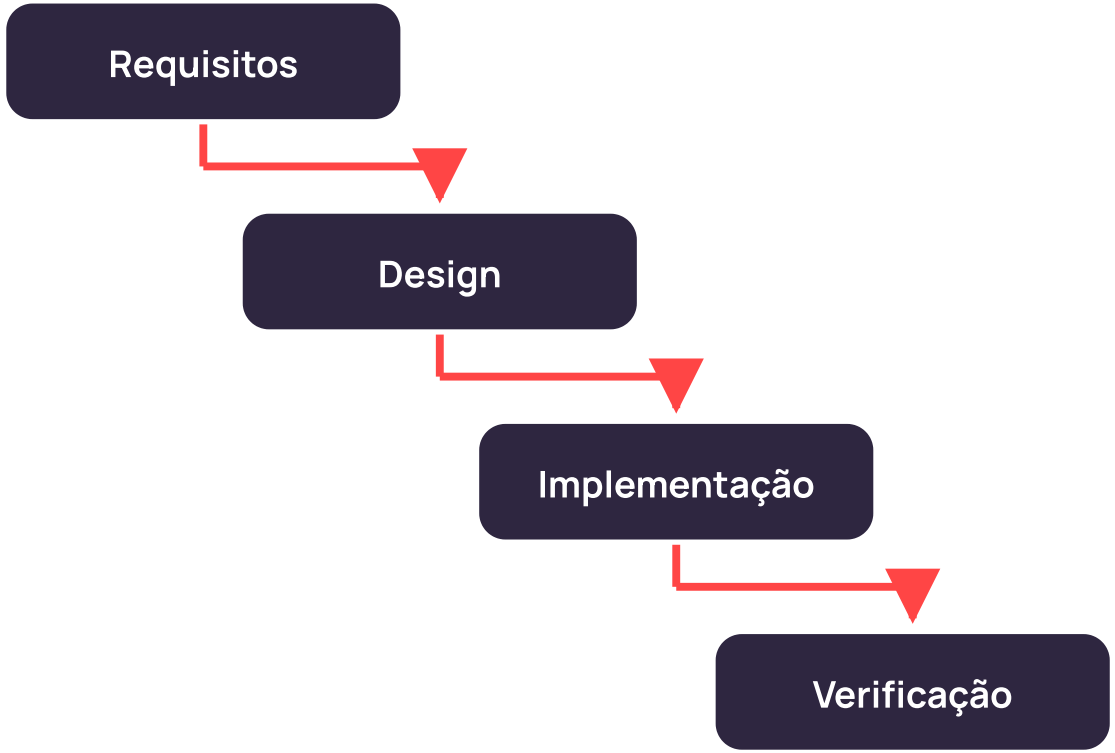
- Decisões viram conversa rastreável.
- Onboarding de gente nova fica mais curto.
- Mudança deixa de doer.

SDLC, as fases do ciclo

O SDLC (Software Development Life Cycle) é a sequência de fases que todo software atravessa. A ordem, a duração e o grau de sobreposição entre elas variam conforme o modelo adotado.

Figura 1. Fluxo de alto nível do SDLC. Cada fase gera artefatos que alimentam a próxima.

Modelo Cascata



Proposto por Royce em 1970, o modelo cascata organiza o SDLC em fases sequenciais. Uma fase só começa quando a anterior termina.

Pontos fortes

É previsível. A documentação sustenta contratos e auditoria.

Pontos fracos

Mudar requisito custa caro. O feedback do usuário só chega no fim.

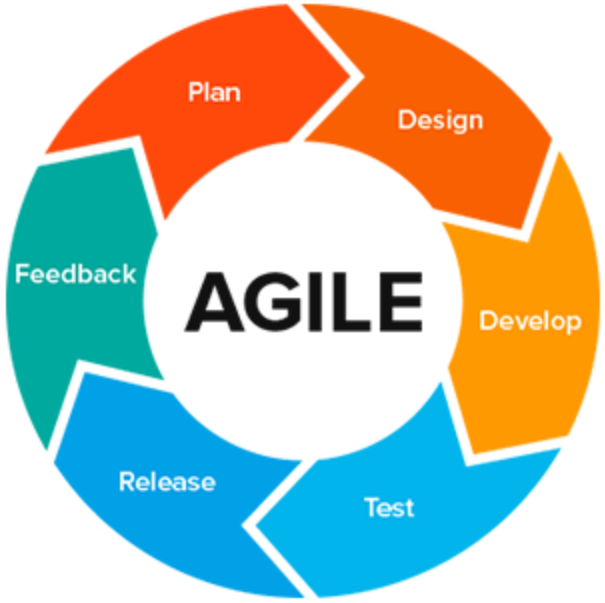
Quando faz sentido

Requisito estável, regulação pesada, sistema embarcado com homologação formal.

Evolução

Modelo em V, que espelha cada fase de construção em uma fase de teste correspondente.

Metodologia Agil



O **Manifesto Ágil (2001)** propôs uma inversão: em vez de fases longas, ciclos curtos com validação frequente. O SDLC continua existindo, mas é percorrido várias vezes, em janelas pequenas.

Os quatro valores do manifesto

Indivíduos e interações mais que processos e ferramentas. Software em funcionamento mais que documentação abrangente. Colaboração com o cliente mais que negociação de contratos. Responder a mudanças mais que seguir um plano.

Frameworks comuns

Scrum, Kanban, XP, SAFe, LeSS.

Ritmo

Sprints de 1 a 4 semanas, entregas incrementais, retrospectivas.

Cascata vs Agil, onde cada um cabe

inteli.

Critério	Cascata	Ágil
Forma do ciclo	Fases sequenciais, cada uma fecha antes da próxima.	Iterações curtas, as fases se repetem a cada ciclo.
Requisitos	Definidos no início, mudança é cara.	Evoluem a cada sprint, mudança é esperada.
Entrega	Produto completo no fim.	Incrementos funcionais ao longo do tempo.
Risco	Concentrado no final, aparece na homologação.	Distribuído, aparece cedo e pode ser corrigido.
Documentação	Extensa, contratual.	Mínima necessária, viva.
Bom para	Regulação pesada, hardware acoplado, contratos fixos.	Produtos digitais, incerteza alta, descoberta contínua.

A pergunta útil não é "qual é melhor?"

É "qual combina com o produto que estamos construindo?". Muitas empresas usam híbridos: cascata para contratos com cliente, ágil para a execução do time.

SALA DE AULA INVERTIDA, 3 MIN EM DUPLAS

Cascata, ágil ou híbrido?

Para cada cenário, decidam: **cascata, ágil ou híbrido**, e justifiquem em uma frase. Não existe resposta única, mas existe justificativa fraca.

1, Firmware de marcapasso

Dispositivo médico implantável, certificação obrigatória pela Anvisa, um bug pode matar o paciente.

O que pesa: risco de vida e regulação.

2, App de delivery novo

Startup quer ver se um novo vertical de mercado emplaca em três cidades antes de escalar nacionalmente.

O que pesa: incerteza e descoberta.

3, Pipeline de ML para fraude

Modelo precisa ser retreinado toda semana, dados mudam, negócio aprova aumento de falso positivo caso a caso.

O que pesa: dados vivos e feedback rápido.

4, ERP bancário regulado

Sistema core do banco, contrato com o cliente trava escopo, auditoria do Bacen exige rastreabilidade completa.

O que pesa: contrato fechado e auditoria.

Papéis no processo de engenharia

inteli.

Um processo só funciona com pessoas claras sobre o que fazem. Os títulos variam por empresa, mas as funções tendem a se repetir.

PO
Product Owner
Representa o cliente, prioriza o backlog e decide o que entra em cada sprint.

SM
Scrum Master
Facilita o processo, remove impedimentos e protege o time do ruído externo.

ARQ
Arquiteto de Software
Define decisões estruturais que são caras de mudar depois.

TL
Tech Lead
Guia técnico do time no dia a dia, traduz arquitetura em prática.

DEV
Desenvolvedor
Implementa, revisa PRs dos colegas e cuida da qualidade do código.

QA
QA, Tester
Pensa testes, encontra problemas antes do usuário e automatiza verificações.

UX
Designer, UX/UI
Projeta a experiência, valida fluxos com usuários e entrega mockups.

OPS
DevOps, SRE
Automação do ciclo, confiabilidade, observabilidade e performance em produção.

Outros papéis aparecem em contextos específicos: Business Analyst, Data Engineer, Security Champion, Release Manager, Engineering Manager.

ALM e SCM, o guarda-chuva e a disciplina

inteli.

ALM, Application Lifecycle Management

Do problema de negócio ao descomissionamento

SDLC, Software Development Life Cycle

Ciclo de engenharia: requisitos, design, implementação, verificação, manutenção

SCM, Software Configuration Management

Identificar, versionar e rastrear cada artefato de software

Git, tags, branches, releases, bill of materials

Pipelines, artefatos assinados, auditoria de mudanças

ALM

É a camada mais ampla. Cuida de portfólio, requisitos, planejamento, execução, operação e fim de vida da aplicação. Conecta engenharia com negócio.

SDLC

É o ciclo de engenharia dentro do ALM. Transforma requisitos em software funcionando, testado e mantido.

SCM

É a disciplina que mantém os artefatos de software rastreáveis: código, dependências, configurações e builds. A Aula 02 mergulha aqui.

ALM por dentro, da ideia ao sunset

inteli.

ALM começa **antes** do código (ideia, requisito, orçamento) e termina **depois** da produção (migração, descontinuação). São cinco frentes, do nascimento ao sunset da aplicação.

01
Portfólio
Qual produto cabe no portfólio da empresa? Roadmap, priorização, orçamento.

`Jira Align, ProductBoard`

02
Requisitos
O que o produto precisa fazer? Histórias, critérios de aceite, dependências.

`Jira, Azure Boards, Confluence`

03
Desenvolvimento
Aqui mora o SDLC. Design, código, testes, revisão, build, release.

`GitHub, GitLab, Jenkins`

04
Operação
Como a aplicação se comporta em produção? Observabilidade, incidentes, SLOs.

`Datadog, New Relic, PagerDuty`

05
Fim de vida
Quando aposentar, migrar ou descontinuar? Plano de saída e preservação de dados.

`Runbooks, ADRs, migração`

Caso real, iFood

O produto nasceu em 2011 como ideia (portfólio), virou um app com poucos requisitos (requisitos), cresceu até ser plataforma de delivery (desenvolvimento), hoje processa milhões de pedidos por dia (operação) e já descontinuou features inteiras como o iFood Shop (fim de vida). O ALM olha essa linha inteira, não só o meio.

SDLC por dentro, o que acontece em cada fase

inteli.

O SDLC é a parte de engenharia do ALM. Cinco fases clássicas, cada uma com entrada, saída e protagonistas. Em times ágeis, elas se repetem a cada sprint; em cascata, rodam uma vez.

Fase	O que entra	O que sai	Quem conduz
Requisitos	Problema de negócio, conversa com cliente, benchmark.	Histórias, critérios de aceite, mockups.	PO, UX, analista
Design	Histórias priorizadas, restrições técnicas.	Arquitetura, ADRs, contratos de API.	Arquiteto, Tech Lead
Implementação	Desenho aprovado, ambiente de dev configurado.	Código em branch, PR aberto, revisão de pares.	Dev, Tech Lead
Verificação	PR aprovado, pipeline verde.	Testes automatizados, regressão, homologação.	Dev, QA
Manutenção	Release em produção, uso real dos usuários.	Correções, melhorias, observabilidade, incidentes.	Dev, SRE, QA

Exemplo, feature de login no Nubank

Requisitos: cliente quer biometria. Design: Tech Lead define WebAuthn. Implementação: dev abre PR. Verificação: QA valida em staging. Manutenção: time observa taxa de sucesso e corrige bugs reportados.

SCM, um primeiro contato

inteli.

Se ALM é o guarda-chuva e SDLC é o ciclo, **SCM é a disciplina que garante que tudo fique rastreável**. Não só o código: também as configurações, dependências e artefatos gerados.

O que o SCM controla

Código, dependências (package.json, requirements.txt), configuração (variáveis de ambiente, feature flags) e artefatos (builds, imagens Docker, pacotes).

Para que serve

Saber quem mudou o quê e quando. Reproduzir a versão exata que rodava em produção no dia do incidente. Voltar atrás com segurança.

Com o que você interage

Git (histórico e branches), tags e releases, pipelines de build, registries de artefatos. Tudo isso forma o bill of materials da aplicação.

Aula 02 mergulha aqui

Estratégias de branching, convenções de commit, proteção de main, assinatura de releases e rastreabilidade ponta a ponta. Por hoje, basta guardar: **SCM é o registro de tudo que entrou na aplicação**.

SALA DE AULA INVERTIDA, 4 MIN EM TRIOS

Encontre os três, ALM, SDLC e SCM

Vou abrir um repositório real no GitHub na tela. Em trios, vocês têm 4 minutos para identificar **um sinal concreto** de cada camada dentro do repositório.

Sinal de ALM

Onde aparece a visão de produto, não só de código? Roadmap, issues agrupadas por épico, milestones com datas, documentos de estratégia.

`Olhe: Projects, Milestones, README de visão.`

Sinal de SDLC

Onde aparece o ciclo de engenharia? Pull requests com revisão, pipeline rodando testes, tags de release, labels de bug/feature.

`Olhe: PRs, Actions, Releases, CHANGELOG.`

Sinal de SCM

Onde aparece o controle de configuração? Arquivo de dependências versionado, lockfile, branch protection, política de commits.

`Olhe: package-lock.json, .gitignore, CODEOWNERS.`

Pipeline de entrega, do commit à produção

inteli.

Cada mudança de código passa por uma esteira automatizada. Em cada estação, ferramentas validam a alteração antes de liberar o próximo passo. Se algo quebra, o pipeline para ali mesmo e avisa quem fez a mudança.

01 — Dev implementa
IDE, branch próprio, testes locais

02 — Commit e push
Git, mensagem clara, referência a issue

03 — Code review
MR aberto, par revisa, discute, aprova

04 — Build, testes e segurança
Compila, roda testes, SAST, SCA

05 — Deploy em homologação
Ambiente igual ao de produção

06 — Validação externa
PO, QA, cliente fazem UAT

07 — Produção
Release liberado, monitoramento ativo

Por que automatizar tudo isso

Toda mudança passa pelo mesmo crivo, sem depender de quem fez o deploy. O tempo entre código pronto e valor entregue cai de dias para minutos, e fica um registro do que rodou, quando rodou e por quê.

Ecossistema CNCF

inteli.

A **Cloud Native Computing Foundation** é a fundação, filiada à Linux Foundation, que hospeda os projetos open source que dão forma ao software cloud-native moderno. É o ecossistema que alimenta as práticas de DevOps que veremos ao longo do módulo.

Exemplos conhecidos

- **Kubernetes**, orquestração de containers.
- **Prometheus**, métricas e alertas.
- **Envoy**, proxy de tráfego de serviço.
- **Fluentd**, pipeline de logs.
- **Argo CD**, entrega contínua GitOps.
- **OpenTelemetry**, instrumentação observabilidade.

Por que interessa ao nosso módulo

- Define padrões abertos para cada fase do ciclo.
- Reduz lock-in de fornecedor.
- Conecta SDLC com operação em produção.
- Serve como base para pipelines de CI e CD.

O landscape tem mais de 200 projetos

Em landscape.cncf.io, o mapa interativo mostra os projetos organizados por função (orquestração, rede, armazenamento, observabilidade, entrega, segurança). Ninguém conhece tudo. Aprender a navegar vale mais que decorar.

On-premises, Cloud e Híbrido

inteli.

Onde o software roda muda o ciclo inteiro. Afeta desenvolvimento, teste, operação e conta no fim do mês.

On-premises

Infra sua, em data center próprio ou alugado.

- Controle total do hardware.
- Dados no perímetro da empresa.
- Custo alto de capex e manutenção.
- Escalar demora, é compra física.

Cloud

Infra como serviço, paga por uso.

- Elasticidade quase instantânea.
- Modelo opex, custos variáveis.
- Automação nativa, APIs tudo.
- Depende de provedor, requer FinOps.

Híbrido

Mistura das duas, com intenção.

- Dados sensíveis on-prem, carga elástica na cloud.
- Integra com regulação local.
- Complexidade operacional maior.
- Pede governança forte.

Na Jacto Drones, os três cenários convivem: firmware roda no drone (edge), plataforma de serviços na cloud e BI pode ter partes on-prem em fazendas com conectividade ruim.

Modelos de serviço, a analogia da pizza

inteli.

Dentro de cloud, o que você opera e o que o provedor opera muda conforme o modelo. Quanto mais o provedor entrega pronto, menos gente sua precisa se preocupar com a infra. A comparação com pizza deixa essa fronteira clara.

ON-PREMISES

Pizza feita em casa

Você faz tudo do zero

- Massa, molho, queijo
- Forno próprio
- Mesa e bebida
- Você assa, serve, lava

IaaS

Pizza congelada

Vem pronta para assar

- Provedor: massa e recheio
- Você: forno, mesa, bebida
- AWS EC2, Azure VM, GCP Compute

PaaS

Delivery em casa

Chega assada na porta

- Provedor: pizza pronta
- Você: mesa e bebida
- Heroku, App Engine, Vercel

SaaS

Jantar na pizzaria

Só sentar e comer

- Provedor: tudo
- Você: consumir e pagar
- Gmail, Salesforce, Notion

Analogia atribuída a Albert Barron (IBM) e popularizada no Brasil por Igor Dsiaducki. Na prática, um mesmo produto usa os quatro modelos ao mesmo tempo: banco de dados no on-prem, APIs em IaaS, frontend em PaaS e Auth0 como SaaS de identidade.

SALA DE AULA INVERTIDA, 2 MIN INDIVIDUAL

Que pizza é essa?

Classifique cada serviço abaixo em **On-prem**, **IaaS**, **PaaS** ou **SaaS**. Anote a resposta e a razão.
Depois, vamos abrir para o plenário em 1 minuto.

1, Amazon EC2

Provisiono a máquina virtual, escolho o sistema operacional, instalo dependências e publico a aplicação.

`Pista: a nuvem me dá a cozinha, não o cozinheiro.`

2, Google Docs

Abro o navegador, escrevo o documento, compartilho o link. Não instalo nada, não atualizo nada.

`Pista: tudo pronto, só consumo.`

3, Heroku

Dou git push e o provedor cuida de build, servidor, banco gerenciado e SSL. Eu só escrevo a aplicação.

`Pista: a plataforma monta o restaurante pra mim.`

4, Servidor no rack do laboratório

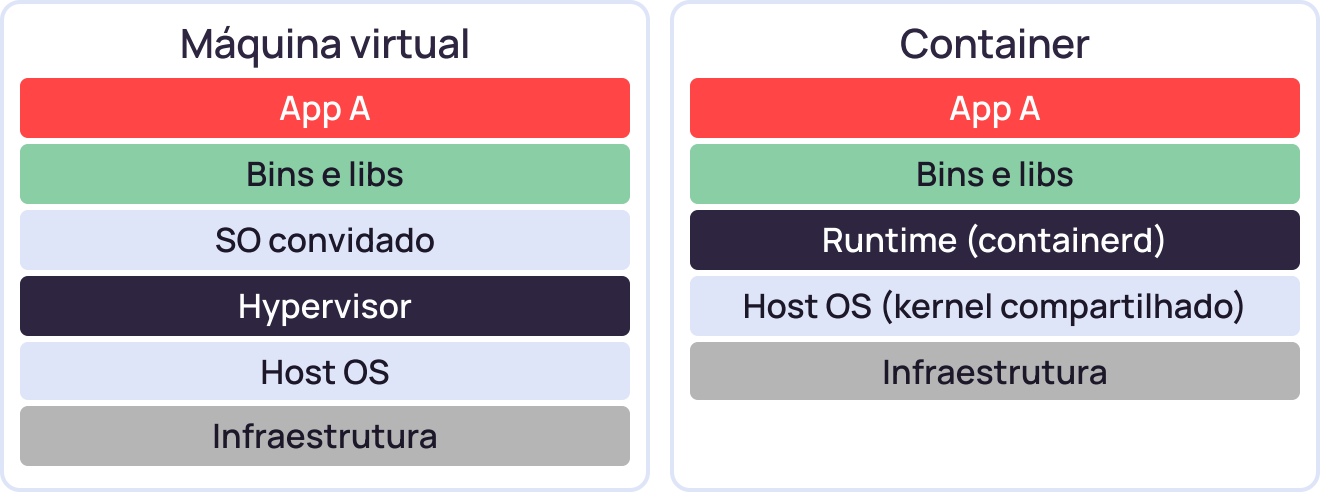
Time interno comprou o hardware, ligou o servidor na sala dos fundos, instalou Linux e subiu o sistema.

`Pista: tudo na mão de casa.`

Containers, leves, portáveis, reproduzíveis

inteli.

Um **container** empacota código e tudo de que ele depende (bibliotecas, binários, variáveis) em uma unidade que roda igual no laptop do dev e em produção. Leva junto o pedaço de SO que precisa, sem arrastar um sistema inteiro.



Por que importa

- **Leveza,** imagens de megabytes sobem em segundos.
- **Isolamento,** cada container tem processos e arquivos próprios.
- **Portabilidade,** mesma imagem em qualquer cloud.
- **Consistência,** some o "na minha máquina funcionava".

O que vem junto

Dockerfile (receita), **registry** (Docker Hub, GHCR, Harbor), **runtime** (containerd, Podman) e o padrão aberto **OCI**, que evita lock-in.

E quando são milhares de containers?

inteli.

Rodar dois ou três containers no laptop é simples. O problema começa em produção, quando cada microserviço tem várias réplicas ligadas 24 por 7 em mais de uma região.

dezenas

docker compose resolve, um script sobe a stack em poucos hosts.

centenas

quem cuida de healthcheck, rolling update, autoscaling, rede?

milhares

impossível na mão. É preciso um orquestrador.

Kubernetes (k8s), o piloto automático dos containers

Projeto open source da CNCF, nascido do sistema interno do Google. Você declara o estado desejado (quantas réplicas, qual imagem, quais recursos) e ele mantém o cluster nesse estado: distribui pods pelos nós, reinicia quando algum cai, escala quando a carga sobe e troca de versão sem parar o serviço. Aulas 05 e 06 do módulo aprofundam.

Governança, mapear workloads antes de escalar

inteli.

Kubernetes resolve o "como rodar". Falta responder quais workloads rodam, quem é responsável, quem paga a conta e quais políticas cada um precisa respeitar. Isso é governança.

WORKLOAD

Carga de trabalho que roda no ambiente computacional para entregar uma função de negócio. Pode ser uma API, um job, um worker de fila ou um batch noturno. Cada workload tem um dono, um SLA, um custo mensal e um conjunto de políticas de segurança associadas.

Mapear é descobrir

- Quantos workloads existem e quem é responsável.
- Quais precisam de alta disponibilidade.
- Quais têm dado sensível (LGPD, PCI).
- Quanto cada um custa por mês.

Governar é aplicar

- Políticas de acesso e segredos.
- Padrões de deploy, observabilidade e backup.
- Revisão de mudanças e auditoria.
- Ciclo de vida, da criação ao desligamento.

Do caos ao catálogo

Em empresas maduras, uma Internal Developer Platform (IDP) trata esse catálogo como produto interno. O dev vê o que existe, pede um novo workload a partir de um template e a plataforma aplica as políticas sozinha. É a ponte para o próximo slide, OpenChoreo.

OpenChoreo, uma IDP para governar workloads

inteli.

OpenChoreo é uma Internal Developer Platform (IDP) open source construída sobre Kubernetes. Materializa o catálogo de workloads que acabamos de ver. Mantida pela WSO2, é a base do serviço gerenciado Choreo.

O que ela entrega

Abstrações de alto nível (Service, Web Application, Scheduled Task, API). O dev declara o workload e a plataforma resolve build, deploy, rede, observabilidade e segredos. Segue o modelo de Platform Engineering da CNCF.

Onde encaixa no ciclo

Cobre do commit ao runtime: organiza pipelines de CI e CD, promove entre ambientes, aplica políticas da organização e publica APIs no marketplace interno.

De onde vem

WSO2 é a empresa por trás (API Manager, Identity Server). **Choreo** é o SaaS operado pela WSO2. **OpenChoreo** é o mesmo produto aberto, para rodar onde a empresa quiser.

Padrão, não ferramenta única

Backstage, Platform.sh, AWS Proton, Port e Humanitec seguem a mesma ideia de IDP. Reconhecer o padrão vale mais do que decorar a marca.

BLOCO 2, 35 MIN

Caso Jacto Drones

Firmware, plataforma de serviços e inteligência de negócios: três frentes sem processo comum. Como resolver?

HANDS-ON, EM SEUS GRUPOS

Desenhe um processo para a Jacto Drones

1. Liste as três frentes da Jacto Drones e 1 risco específico de cada uma quando operam sem processo compartilhado.
2. Proponha um SDLC comum com pelo menos 5 fases nomeadas. Decidam juntos: cascata, ágil ou híbrido?
3. Em cada fase, indique quem faz (firmware, plataforma, BI), o que entrega e como as três frentes se integram.
4. Marque 2 pontos do ciclo onde ALM ajuda a reduzir ruído entre as frentes.
5. Apresente em 2 min: o que muda quando existe processo unificado?

Checagem rápida, 1 de 3

inteli.

Qual afirmação descreve melhor a relação entre SDLC, ALM e SCM?

- SCM engloba o ALM, que é apenas uma técnica de desenvolvimento clássico.
- Os três termos se equivalem, muda só o fornecedor que usa cada sigla.
- ALM cobre o ciclo da aplicação, contém o SDLC e inclui o SCM como disciplina.
- SDLC pertence ao ágil, ALM à cascata, e SCM é ferramenta de build.

Checagem rápida, 2 de 3

inteli.

O que diferencia Integração Contínua (CI) de Entrega Contínua (CD) em um pipeline?

- CI e CD descrevem a mesma prática, muda só o nome que cada fornecedor usa.
- CI valida o commit em build e testes, CD leva o artefato aprovado a ambientes.
- CI roda apenas em produção e CD roda em homologação, ambos pedem aprovação humana.
- CI gera binários para Windows e CD gera para Linux, cada um em seu pipeline.

Checagem rápida, 3 de 3

inteli.

Por que Kubernetes, e não só containers, virou padrão em produção de escala?

- Ele orquestra muitos containers, cuida de escala, reinício e rede sem script.
- Kubernetes substitui o container e dispensa a imagem Docker no deploy.
- Sem Kubernetes, containers só rodam em notebook e não funcionam em Linux.
- Kubernetes criptografa o código e atua como ferramenta de segurança.

Auto-estudo para a próxima

inteli.

Placeholder, aguardando lista oficial dos auto-estudos da Aula 02.

Leitura

Capítulo introdutório sobre SCM e controle de versão distribuído.

Vídeo

Vídeo curto comparando GitFlow, Trunk-Based e GitHub Flow.

Entrega

Responder no AdaLove a atividade de autoestudo da Aula 02.

inteli.

Até a próxima

Aula 02, Gestão de configuração (SCM), 27/04